

NLP in Practice

Unit 4 Final Project: Build an Agent with LangGraph

Albert School of Business & Data



Objective

Build a small but functional agent using LangGraph. The agent must reason about a user request, decide between multiple actions, call tools, maintain state, and respond. The topic is entirely up to you: pick a problem you find genuinely interesting. The grade rewards the soundness of your design and implementation, not the ambition of the topic.

Important. This is the final project of the course and ties together everything from Units 1 to 4: LangChain composition, prompts and parsers, RAG (if you choose to use it as a tool), and LangGraph control flow. It is also intentionally open. There is a base specification you must meet to pass; beyond that, you decide how deep to go.

Project philosophy

This project is intentionally open-ended. The grade is on a 0-10 scale. Meeting the mandatory requirements cleanly is worth 5/10; the “Going further” section is how you climb above that. A clean, minimal agent that works end-to-end will score better than a sprawling, half-broken one. Start small, get the loop running, then add depth where it matters most for your use case.

A simple agent that works reliably is worth more than an ambitious one that breaks under inspection.

Mandatory requirements (to pass)

Your submission must satisfy all of the following. If any of these is missing, the project does not pass the technical bar regardless of how polished the rest is.

- **LangGraph:** the agent must be built as a `StateGraph`, not as a plain `LangChain` chain. A linear prompt | LLM does not qualify.
- **Typed State:** a `TypedDict` (or equivalent) holding the fields your agent actually uses. Only include fields the graph reads or writes.
- **Nodes:** at least three nodes, each as a function from `State` to `PartialState`.
- **Conditional edge:** at least one decision point where the next step depends on the `State` (either the LLM decides via tool-calling, or your code routes based on a field). A graph with only sequential edges is a chain in disguise.
- **Tools:** at least two tools the agent can call. A tool can be a web search, a RAG retriever, a database query, a calculator, an API call, a file reader, or anything else that gives the agent reach beyond pure text generation.
- **Streamlit UI:** a working interface a user can interact with. It does not need to be beautiful, but it must let someone use the agent without reading the code.
- **Evaluation:** design a small test suite, define success criteria, and measure your agent's success rate. Identify and document failure modes.

- **README:** one to two pages explaining what the agent does, the graph design (a diagram is welcome), how to run it, the key design choices, and three example interactions you tested with their outcomes. Since the project is evaluated only on the written submission, the README is the main document the grader reads.

Going further

These items are graded as Part B of the rubric and are what lets the project score above 5/10. Pick what fits your project: two or three extensions executed well are worth more than a long list done superficially.

- **Persistence:** add a checkpoint so the agent's state survives a page reload or a server restart.
- **Human-in-the-loop:** use `interrupt_before` to pause the graph at sensitive points and ask the user to approve before continuing.
- **Streaming:** stream intermediate steps (which node is running, which tool was called, with what arguments) to the UI so the user can follow the agent's reasoning.
- **Sub-graphs or multi-agent:** decompose the work into specialised agents that hand off tasks to each other.
- **RAG as a tool:** integrate the retrieval pipeline from Unit 3 as one of the agent's tools.
- **MCP:** connect to an external MCP server to expose real-world tools to the agent.
- **Observability:** trace every node, decision and tool call. LangSmith works well; a simple structured log is also acceptable.
- **Safety harness:** enforce a token budget, a max number of loop iterations, and a fallback path when a tool fails or the LLM goes off-rails.

Suggested project ideas

These are starting points, not a closed list. You may propose anything you can defend as a useful agent. If you are not sure your idea fits the scope, ask before building.

- Personal research assistant: web search + RAG over your own notes + summarisation, with a conditional decision on whether to search externally or stay local.
- Code review agent: reads a diff or a file, runs static checks, queries documentation, and proposes improvements with rationale.
- Travel planner: searches destinations, computes itineraries, holds preferences in state, and asks for human approval before committing.
- Personal finance assistant: queries a bank export (CSV), computes budgets, flags anomalies, and suggests savings actions.
- Meal planner: takes dietary preferences, generates a week of meals, builds the shopping list, and adapts when the user rejects a meal.
- Study buddy: builds flashcards from your notes, runs quiz sessions, tracks performance per topic, and suggests what to review next.

- News triage agent: fetches articles, classifies them by topic, summarises the relevant ones, and escalates anything matching your interests.
- Email assistant: classifies incoming messages, drafts replies, and pauses for human approval before sending anything externally.
- Customer support agent over a product manual: classifies the query, retrieves the relevant section, answers if confident, escalates otherwise.

Required stack

- **LangGraph** for the agent.
- **An LLM via API:** Groq, OpenAI, Anthropic, or any provider with a LangChain integration. Free-tier Groq is sufficient for the mandatory part.
- **Streamlit** for the UI.
- **Other libraries** as needed for your tools (Chroma if you use RAG, requests for APIs, pandas for tabular tools, and so on).

Deliverables

A Git repository or a ZIP archive containing:

- All the code (the graph, the tools, the Streamlit app).
- `requirements.txt` with pinned versions.
- **README.md** as described above (what the agent does, graph design, how to run, key design choices, three example interactions).

Do not include API keys, large data files, virtual environments, or local caches in the submission.

Individual or pair

Same format as the Unit 3 assignment. Projects may be completed individually or in pairs of two. Both members of a pair receive the same grade

Grading

The project is graded on a 0-10 scale. The grade is built from two parts: a base grade from the mandatory work, and an extension grade from the work beyond it.

Part A: base grade (up to 5/10)

Built from the mandatory requirements. A submission that meets every mandatory item with sound design and clean code scores 5/10. That is the pass mark and a respectable grade for a working, well-built agent. Within Part A, the grader weighs four areas:

- **Problem definition and agent design (20%):** clarity of what the agent is supposed to do and for whom, justification that an agent (rather than a chain or pipeline) is the right architecture, quality of the State design.

- **LangGraph architecture (30%):** correct use of StateGraph (nodes, edges, at least one meaningful conditional edge), reducers used when fields accumulate, the graph compiles and runs end-to-end on a realistic input, every node and edge earns its place.
- **Tools and integration (25%):** at least two well-chosen tools, clear tool boundaries (one responsibility per tool), error handling when a tool fails, clean integration with the graph.
- **Implementation quality and documentation (25%):** code clarity and reproducibility, a Streamlit UI that a non-author can use without reading the code, a README that explains design (with a graph diagram and worked examples), a self-contained submission.

Part B: extension grade (up to +5)

Built from the “Going further” section. This is what determines how high above 5/10 the project scores. A perfect Part A with no extensions scores 5/10. Climbing above 5 requires depth here. The grader weighs:

- **Coherence:** extensions should serve the project's purpose. Gratuitous complexity is not rewarded.
- **Impact:** extensions that visibly improve the agent's behaviour, robustness, or usefulness are valued over those that only show off a feature.
- **Reflection:** the README should explain what each extension contributes and what trade-offs it introduces.
- **Creativity:** original ideas, unexpected tool combinations, or a novel use case that goes beyond the suggested project list are rewarded.

Submission

- **Submission deadline:** to be confirmed.

Hints

- Start by drawing your graph on paper. State at the top, nodes as boxes, edges as arrows. If you cannot draw it in one minute, the design is too complex.
- Build the minimum viable graph first (one node, one tool, no routing) and check it runs. Then add the second tool. Then add the conditional edge. Resist the urge to design the full graph before any of it runs.
- For the State, include only fields the graph actually reads or writes. Empty fields are a sign the state was designed by aspiration rather than by need.
- The conditional edge is the part students get wrong most often. Test it with at least three inputs that each route differently. If a branch is never triggered, it is dead code.
- Streamlit reruns the whole script on every interaction. Cache your model and your compiled graph with `@st.cache_resource`. Recompiling the graph on every keystroke is the easiest way to make the app feel broken.
- If you use a checkpoint, test that the state actually survives a page reload. Do not just trust it.

- Two clean tools beat five sloppy ones. The same applies to nodes.
- Document your prompts. The system prompts you give to each node are part of the design and should appear in the README.

Final remarks

This project is your chance to build something you would actually want to use. Take the topic seriously, take the design seriously, and ship something honest. A working agent that does one thing well is the goal; everything else is decoration.

Agent = LLM + Loop + Tools + Harness. The four units of the course gave you each of those pieces. The project asks you to put them together.